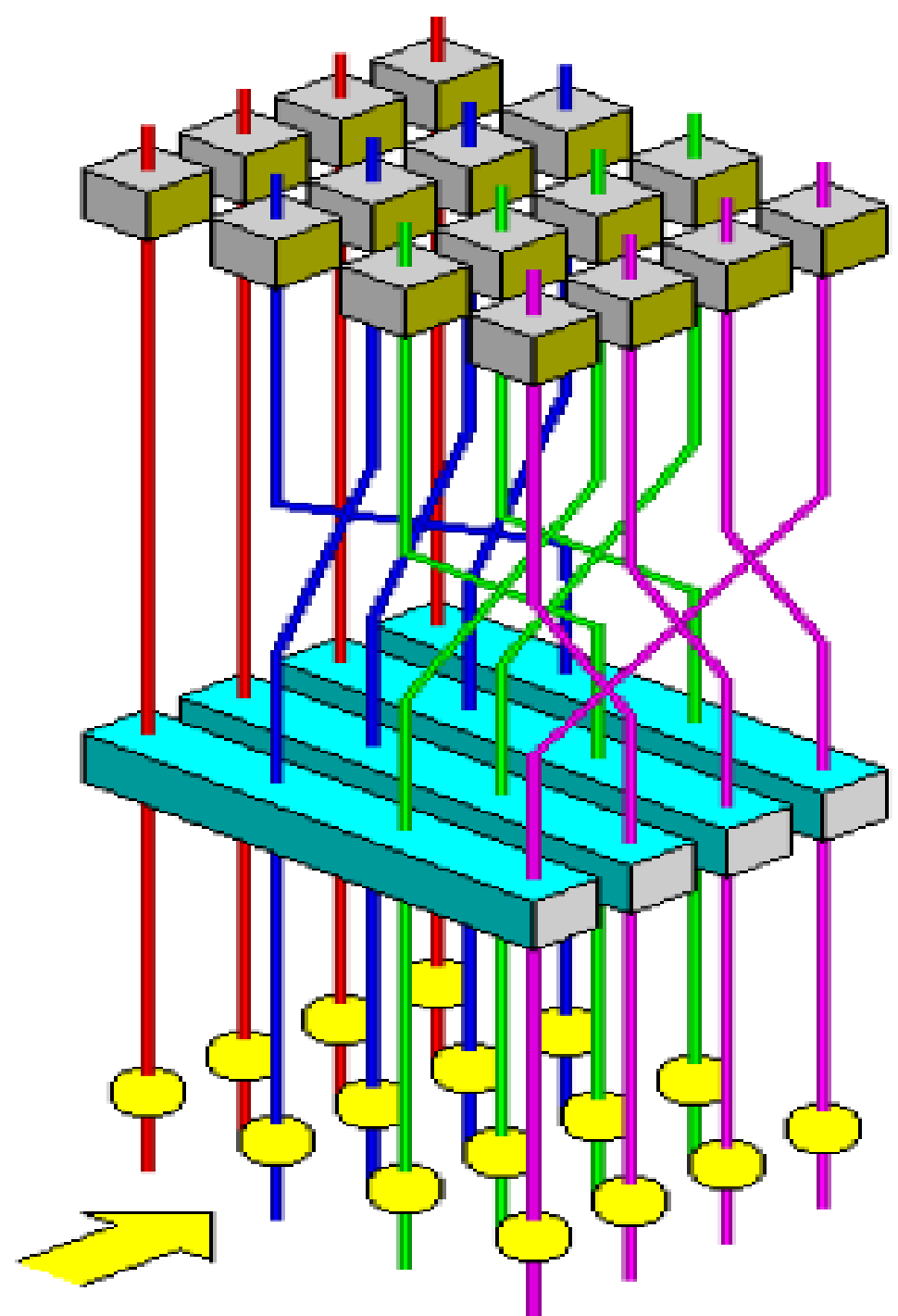


Processor Design Project

Group 24

12-06-202
6



Introduction

- Data security is critical across embedded systems, IoT, and communications infrastructure
- Encryption is the primary mechanism
- AES is the global standard

The primary goal of this project:

- implement AES acceleration on a RISC-V core efficiently in hardware



Where is AES used?

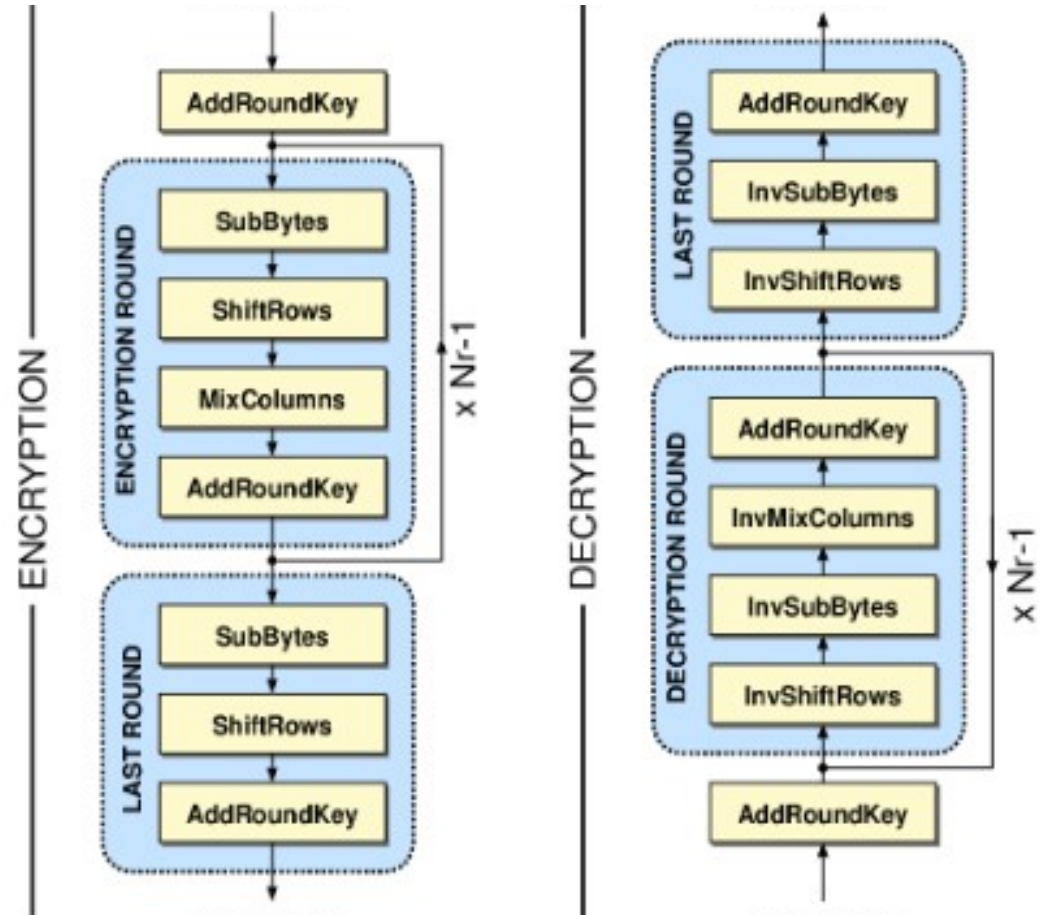
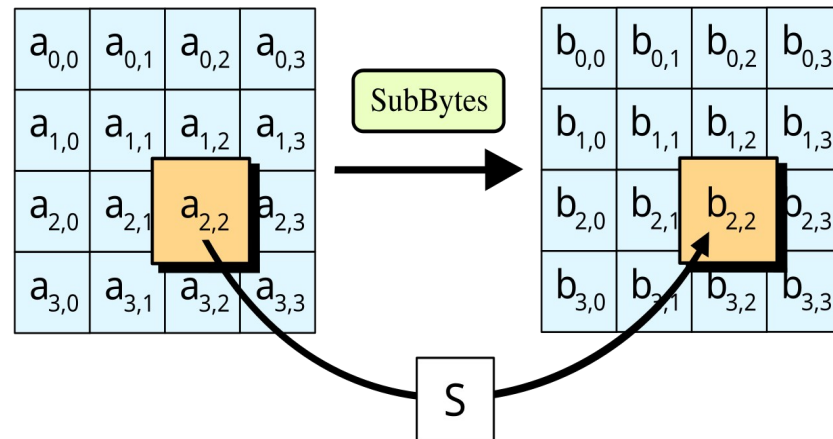


- HTTPS/TLS
- Wi-Fi (WPA2/WPA3)
- File encryption
- Cloud storage
- VPNs
- USB drives & SSDs
- Smart cards & passports
- IoT devices

Advanced Encryption Standard (AES)

What is AES?

- Symmetric block cipher — same key for encryption and decryption
- Operates on a **4×4 matrix of bytes** called the **state** (= 128 bits)
- Key is expanded into **11 round keys** via the key schedule



Advanced Encryption Standard (AES)

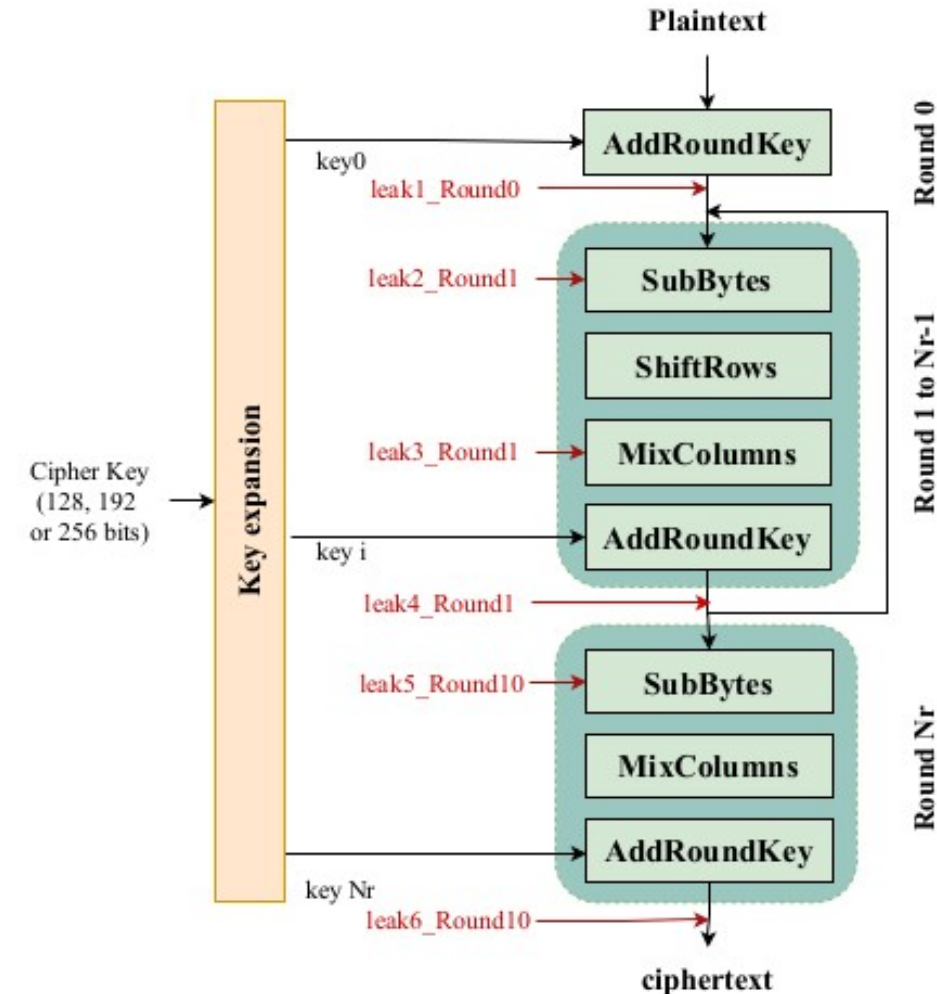
How does AES work?

Initial Round Plain text XOR'd with round key 0

Middle Rounds × 9

- **SubBytes** — Byte substitution via S-box (confusion)
- **ShiftRows** — Cyclic row shifts to spread data across columns
- **MixColumns** — Column mixing over $GF(2^8)$ (diffusion)
- **AddRoundKey** — XOR with current round key

Final Round (Round 10) Same as middle rounds but **no MixColumns**



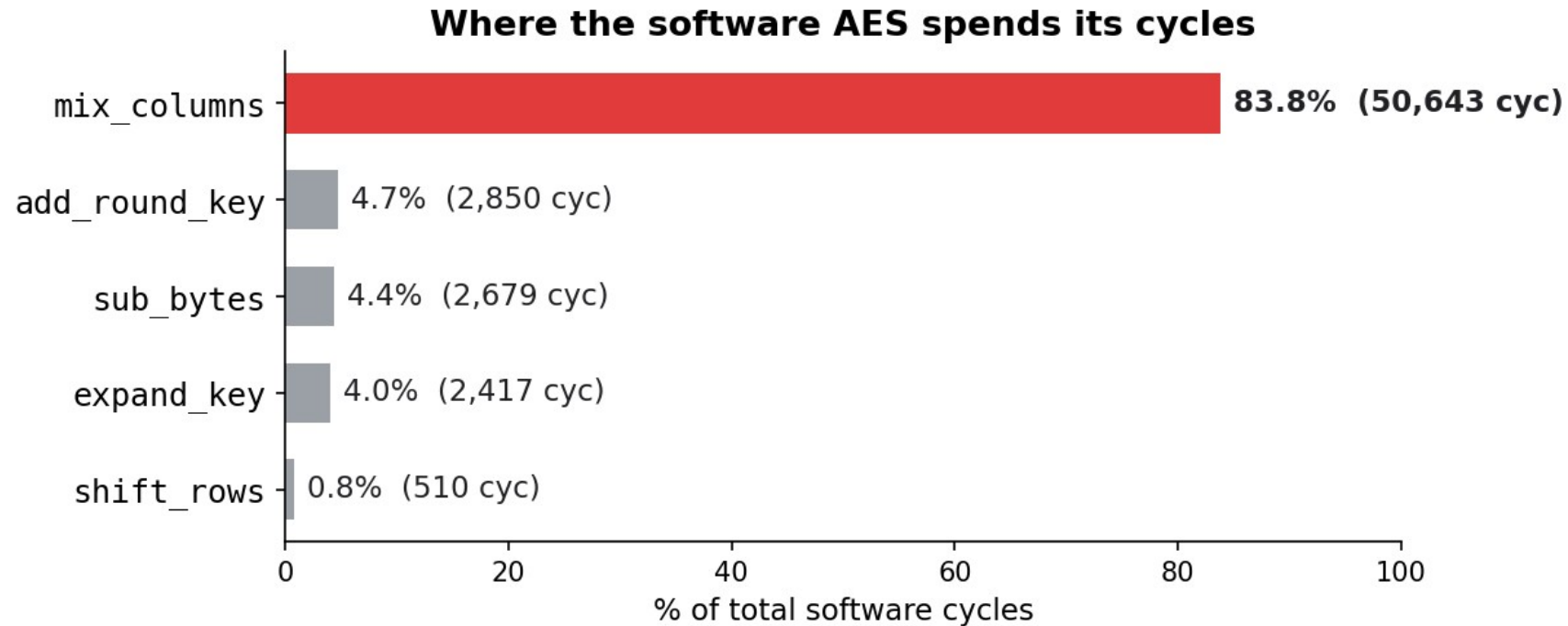
Baseline software implementation

- Pure C AES-128 on the unmodified RISCY core: standard loads, stores, XORs and shifts.
- Table-based S-box and gf_mult MixColumns, all from data memory; no hardware acceleration.

Resource	Count
Cycle Count	61184
LUTs	4644
Registers	2163
Muxes	284

61,184 cycles per block, almost all spent in MixColumns (next slide). This is the bottleneck the hardware targets.

Baseline Bottlenecks



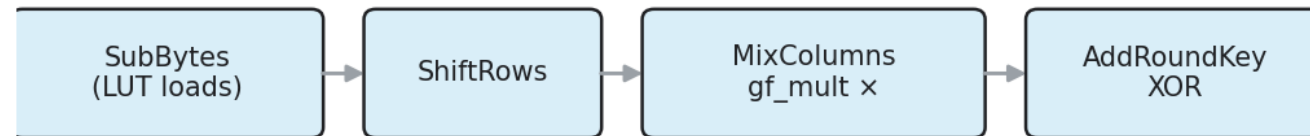
MixColumns (the `gf_mult` inner loop) is 83.8% of all cycles, so it is the single thing worth accelerating in hardware.

How does hardware help?

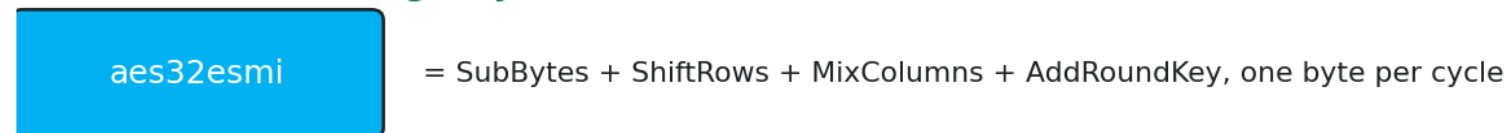
The 186-instruction software `mix_columns` collapses to a few `aes32esmi` calls (no `gf_mult`).

- Runs entirely on registers: no BRAM loads for the S-box or MixColumns.
- `aes32esmi` = SubBytes + ShiftRows + MixColumns + AddRoundKey, in one cycle.

Software round (≈ 186 instructions)



Hardware round (single-cycle instruction)



Performance after implementation of AES hardware support

aes32esmi does **SubBytes + ShiftRows + MixColumns + AddRoundKey** in a single cycle, removing the **mix_columns** bottleneck that was 83.8% of software runtime.

Version	Cycles	vs software baseline
Software AES (baseline)	61,184	1.0×
+ aes32esmi / aes32esi hardware	6,260	9.8x faster

The looped hardware version (6,260) is the starting point for the later optimisations: custom loop-unroll pass → 4,800, parallel DOM S-box → 4,104 (covered on the compiler and RTL slides).

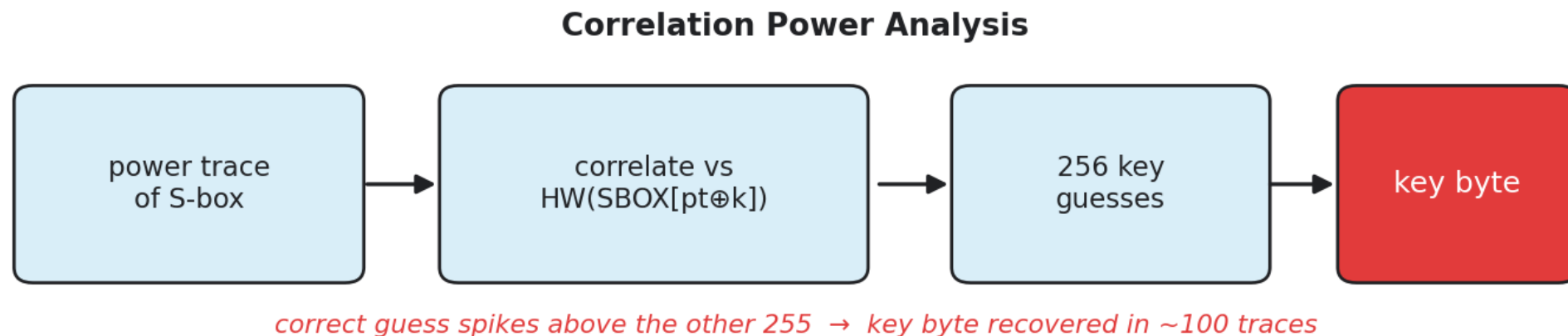
Vulnerabilities to side channel attacks

The Zkne S-box and MixColumns switch a data-dependent number of bits, so power leaks the byte being processed.

Correlation Power Analysis turns that leakage into the key: about 100 traces on our unprotected core.

Countermeasure: Domain-Oriented Masking (DOM). Secrets are split into two random shares, recombined only inside registered gates, so no wire (or glitch) tracks the real value.

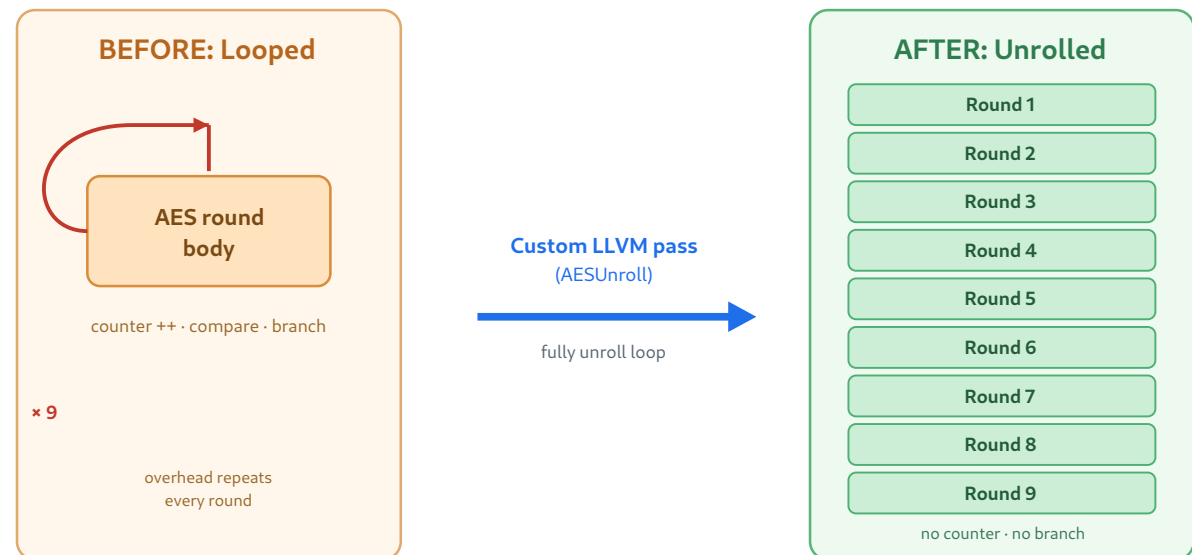
Follows the TU Delft secure-Zkne paper of Kassimi, Aljuffri, Hamdioui and Taouil (2026).



Compiler Optimizations: Custom LoopUnroll Pass

What was done?

- Wrote our own LLVM compiler pass to fully unroll the 9-round AES loop
- Calls LLVM's unroll routine directly instead of using a built-in flag
- Flattens all 9 rounds into straight line code
- Detects the loop by spotting the AES instructions inside it, so it survives inlining



Compiler Optimizations: Custom LoopUnroll Pass

Why we did it?

- The looped version wastes cycles on loop overhead every round (counter, branch, condition check, key pointer reload).
- Built in unrolling is cost model driven and often refuses to unroll, especially when optimizing for size.
- Wanted full control to force unrolling on exactly the target loop.
- Goal was to build the optimization myself, not just flip a compiler switch.

Compiler Optimizations: Custom LoopUnroll Pass

Results

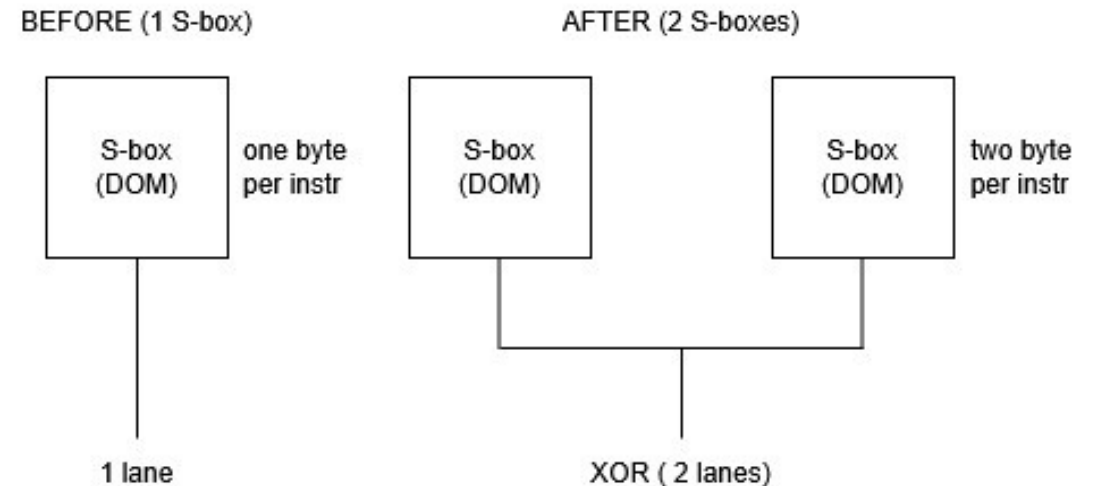
- Brought the software from 6,260 to 4,800 cycles, about 23 percent faster
- Same correct ciphertext, guarantees the loop is always fully unrolled

Build	Unroll Method	Cycles	Vs. Baseline
GCC -Os (looped)	none	6260	-
GCC -Os + #pragma unroll	Source pragma	6067	-3%
GCC -O2 + - funroll-loops	Build-in flag	5509	-12%
Clang -Os + Custom pass	AESUnroll	4800	-23%

RTL Optimization: Parallel DOM S-box

What was done?

- Added a custom instruction that runs two DOM protected AES S-boxes in parallel
- Each instruction now processes two-byte lanes of a column instead of one
- Built a new execution unit in the core (two S-boxes side by side) plus decoder and pipeline changes
- Software XORs the two halves together to finish each column
- Kept the original single S-box path intact as a fallback

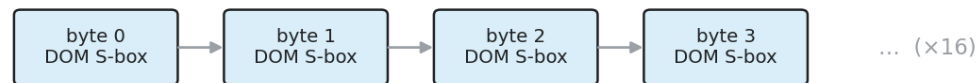


RTL Optimization: Parallel DOM S-box

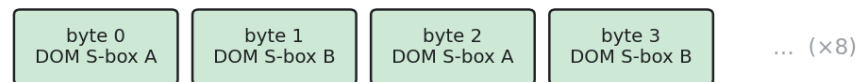
Why we did it?

- The DOM S-box is the real bottleneck: every byte pays a fixed 5 cycle latency and stalls the whole pipeline
- The original design did the S-box one byte at a time, so the 16 bytes per round were fully serial
- Running two S-boxes at once halves the number of instructions per round
- The masking (side channel protection) had to stay fully intact

Original (serial): 16 bytes, one S-box at a time → 16 × 5-cycle ops



Parallel 2-lane: two DOM S-boxes at once → 8 × 5-cycle ops (half the steps)



Masking (DOM) stays fully intact; one instruction now finishes two byte-lanes.

RTL Optimization: Parallel DOM S-box

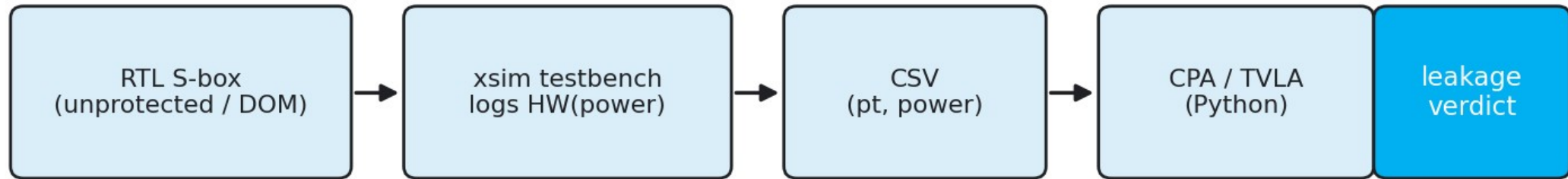
Results

Stage	Cycles	Vs. Original
Original	6260	baseline
Custom Unroll Pass	4800	23% faster
Parallel DOM S-box	4104	34% faster

Resource	Count
Cycle Count	4121
LUTs	11987
Registers	10412
Muxes	289
DSP	5

Validation Framework: Simulation-Based Power Analysis

How we proved the masking works, in simulation and on hardware



Power proxy = Hamming weight of the captured S-box result, logged per trace by a SystemVerilog testbench in Vivado xsim. The same rig drives the unprotected and the DOM S-box, so the before/after is clean.

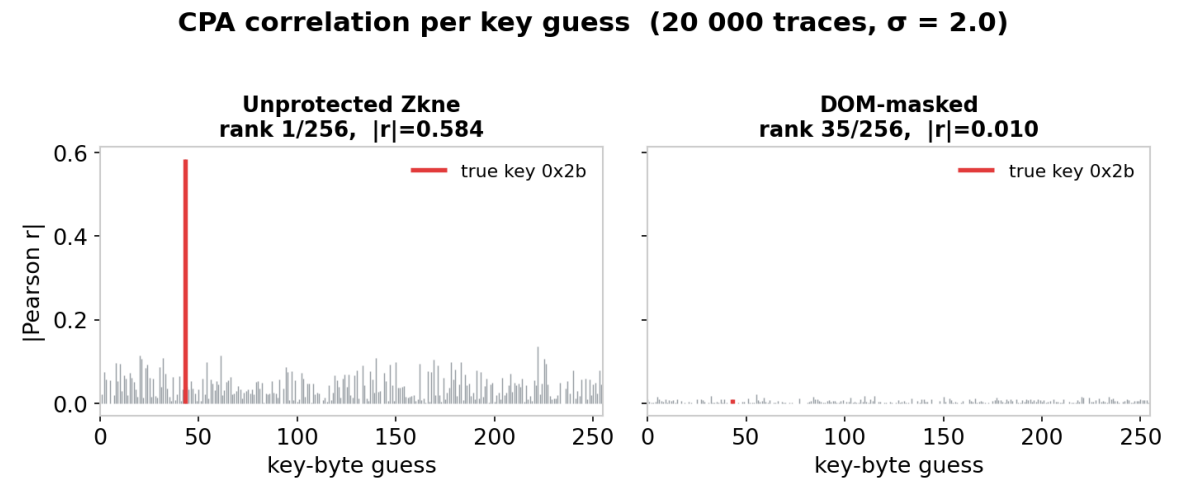
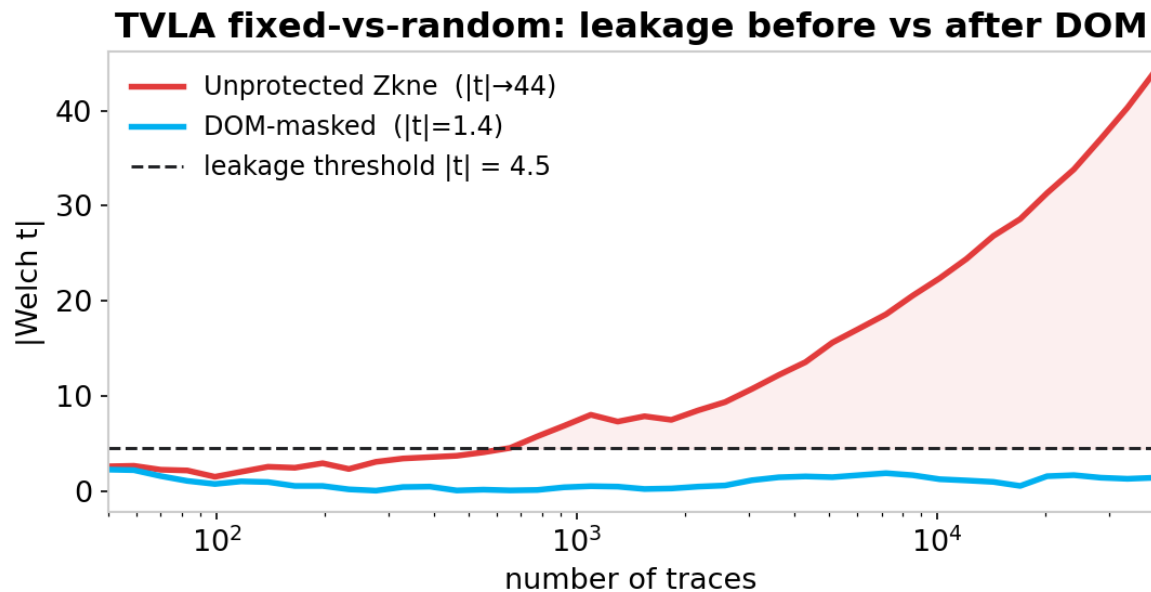
CPA: correlate power with $\text{HW}(\text{SBOX}[\text{pt XOR guess}])$ over all 256 guesses, 20 000 traces.

TVLA: Welch fixed-vs-random t-test, 40 000 traces; $|t| > 4.5$ means first-order leakage.

Simulation isolates the data-dependent switching with no analog noise, alongside Rishi's CW305 hardware validation.

Validation Results: DOM Defeats the Attack

Same rig, same noise ($\sigma = 2.0$). The only change is the S-box, and the leakage disappears.



Metric	Unprotected	DOM-masked
CPA true-key rank	1 / 256	35 / 256
CPA top $ r $	0.58	0.022
TVLA max $ t $	44.0	1.4
Crosses $ t = 4.5$?	Yes (9.8x)	No (0.3x)

Hardware Validation: CW305 Setup

From simulation to silicon - Validation on CW305 Artix-7 FPGA with CV32E40P RISC-V soft core

Hardware setup

- CW 305 Artix-7 100T with CV32E40P RISC-V soft-core
- ChipWhisperer-Lite scope, 50 MHz clock, ADC at 200 MHz
- Gain: 30 dB
- Three bitstreams tested: AES_100t (reference), Baseline RI5CY, DOM-protected RI5CY

Attack Method

- CPA: sbox_output leakage model and last_round_state_diff leakage model
- Window: samples 0-80 (round 1 S-Box)
- TVLA: Welch t-test, fixed vs random PT
- Threshold: $|t| > 4.5$

Hardware Validation Results: DOM defeats the attack

Real silicon confirms simulation – leakage disappears with DOM masking

Metric	AES_100t	Baseline RI5CY	DOM Protected RISCY
CPA bytes recovered	16/16	0/16 (Need more traces)	0/16 (Protected)
CPA max correlation	0.236 (strong)	0.21, PGE dropping	0.07 (noise floor)
TVLA max t	49.27 (10.9x threshold)	4.69 (marginal, 1 sample)	3.98 (below threshold)
Leakage detected	Yes	Marginal (1 sample)	No

Key Findings:

- CPA successfully breaks unprotected hardware AES with correlation = 0.236
- DOM masking bring correlation to 0.07 and TVLA t-value below the leakage threshold, confirming first-order SC resistance on real hardware.

Future Work

The observation:

- Same operation four times
- This results in:
 - Additional execution overhead
 - Repeated instruction fetch and decode.
 - Increased instruction count. .
- Independent, highly regular pattern.
- Places a limit on the throughput

```
aes32esmi t0,t0,a4,0  
aes32esmi t0,t0,a5,1  
aes32esmi t0,t0,a6,2  
aes32esmi t0,t0,a7,3
```

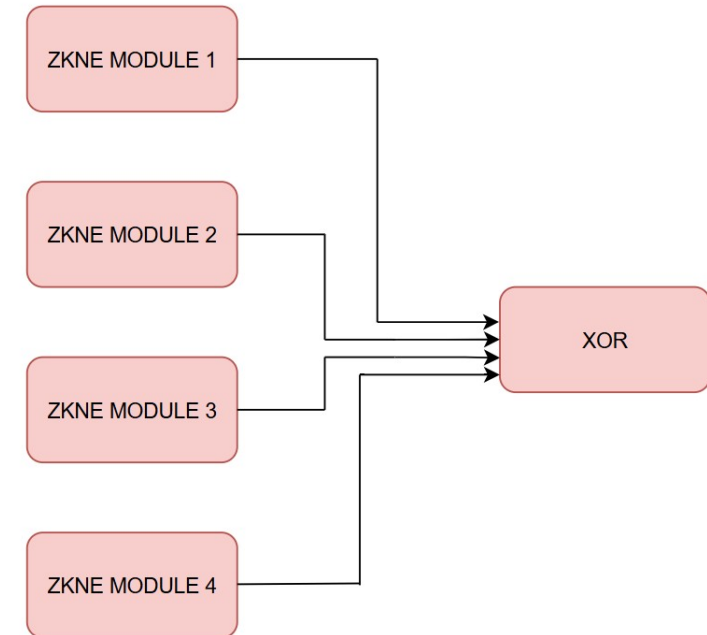
4 instructions required for a single round. A consequence of the bit width of the registers

The Super Instruction



The Idea:

- i) Introduce a super instruction
- ii) Replicate AES32/Zkne datapath four times
- iii) Execute in parallel within a single instruction.
- iv) Combine the results internally and write back the final 32-bit result.

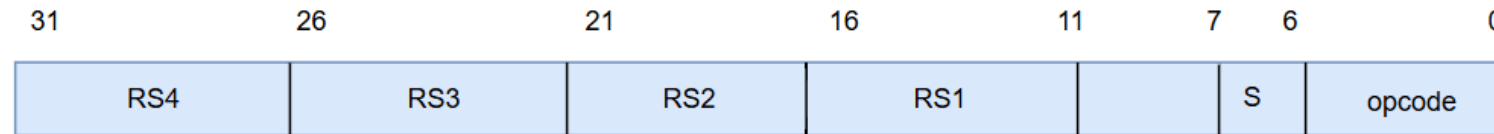


Proposed Block Diagram

The Super Instruction

Architectural Changes

i) New opcodes and instruction formats



Proposed Instruction Format

ii) Modifications to the Register File (or) Decode Logic

iii) Replicate the Zkne hardware

iv) Software Support for the Instruction

The Super Instruction

Limitations

Opcode space pollution

Custom-0 opcodes consume scarce encoding space and may clash with future standard extensions.

Register-file pressure

Reading four source registers needs extra read ports or a 2-cycle read, complicating the pipeline.

Higher hardware utilisation

Replicating the Zkne datapath four times costs significant extra LUTs and flip-flops.

Narrow workload applicability

The fused op only accelerates AES; that area sits idle for any other workload.

THANK YOU !!

